

Introduction to Unity

How to download Unity and understand the basics of the editor

1. What is Unity?

Unity is a game engine and editor used to make 2D games, 3D games, and interactive apps. You build your game visually by placing objects in a scene, and later you add behaviour to them with small C# scripts. A single Unity project can be exported to Windows, Mac, Android, iOS, the web, and consoles.

You do not need any programming to start exploring. This guide covers installing Unity and learning what each part of the editor does.

2. How to download and install Unity

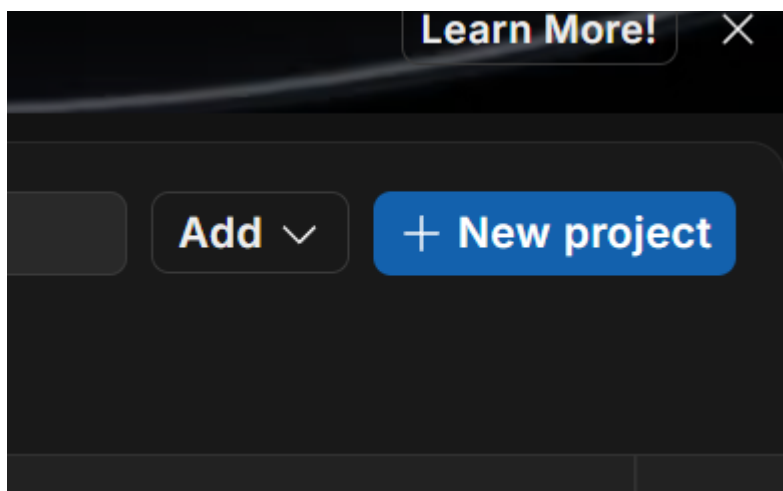
Unity is installed in two parts: Unity Hub, a launcher that manages your projects and Unity versions, and the Unity Editor, the program you make games in.

1. **Download and Install Unity Hub.** Go to unity.com/download and download Unity Hub for your system (Windows, macOS, or Linux).
2. **Install an Editor version.** In the Hub, click Installs in the left sidebar, then Install Editor. Under Official releases, choose a version marked LTS (long-term support). LTS versions are the most stable choice for beginners.
3. **Choose modules.** Select any extra modules you need (it's ok if you select none, cuz most of them can be installed later on), then click Install.

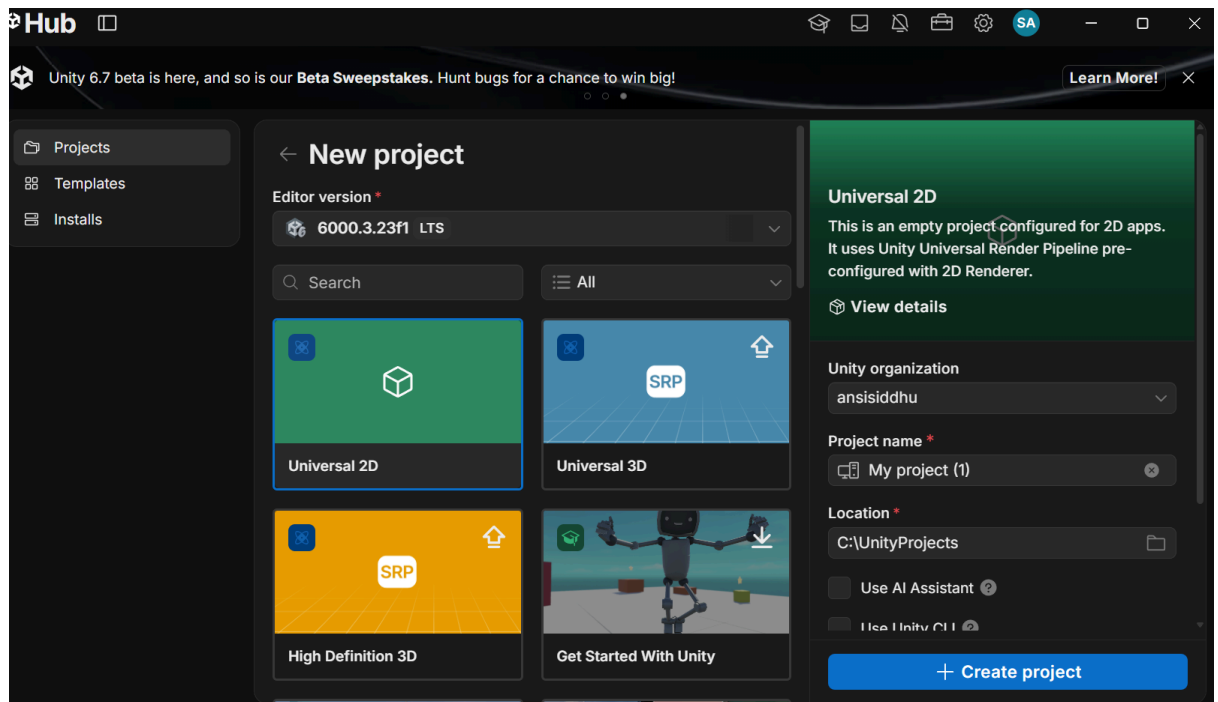
A code editor: Unity scripts are written in C#, so you will eventually want a code editor. The Hub can install Visual Studio for you, and Visual Studio Code and JetBrains Rider also work.

3. Creating your first project

1. In Unity Hub, open Projects and click New project.



2. Choose your Editor version, then a template. Pick a Universal 2D for 2D games or a Universal 3D for 3D games
3. Enter a project name and location, then click Create project.



The first launch takes a few minutes while Unity sets things up. After that, the project opens quickly.



4. The Unity editor layout

The Unity window is divided into panels, and each panel has one job. These are the main ones. Panels can be dragged and resized, so your layout may look slightly different.

Hierarchy

The Hierarchy window in Unity is a text-based panel that displays every GameObject currently present in your loaded Scene.

Inspector

Shows the details of whatever you have selected. It lists the object's components, and each component has settings you can edit, such as position, colour, or physics values. Most of your tweaking happens here. The checkbox next to an object's name turns the object on or off.

Scene view

Your workspace. Here you look at the world from any angle and place, move, rotate, and scale objects. What you see here is for you, not for the player.

Game view

Shows what the camera sees, which is what the player will see. When you press the Play button, the game runs here.

Project window

Holds every file in your game inside the Assets folder: scripts, images, sounds, prefabs, and scenes. The Hierarchy shows what is in one scene, while the Project window shows everything you own. Drag an asset from here into the Scene view or Hierarchy to add it to your game.

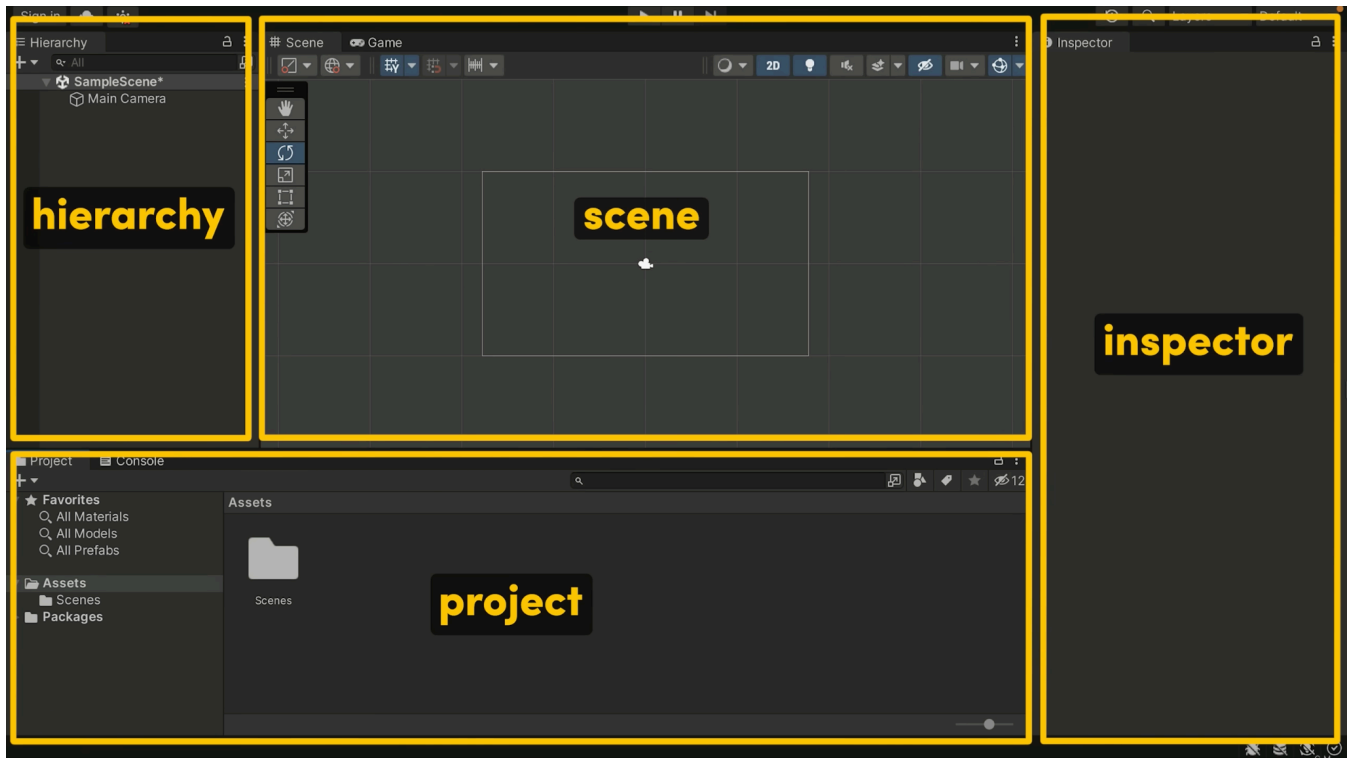
Console

Where Unity shows messages, warnings, and errors. When something goes wrong, check the Console first. It tells you what the problem is and where.

Toolbar

The strip along the top of the window. It holds the tools for moving, rotating, and scaling objects, and the Play, Pause, and Step buttons.

Important: Changes you make while the game is in Play mode are not saved. When you press Stop, everything returns to how it was before. Make your edits while the game is stopped.



5. Core ideas to know

Term	Meaning
Scene	A level or screen: a saved collection of objects. A game usually has several, such as a menu and levels.
GameObject	Anything that exists in a scene. On its own it is just an empty container.
Component	A piece of behaviour attached to a GameObject, such as how it looks, its physics, or its script. A GameObject is built from components.
Transform	A component every GameObject has. It stores the object's position, rotation, and scale.
Prefab	A reusable template of a GameObject. Build something once, save it as a prefab, and place as many copies as you like. Editing the prefab updates all copies.
Asset	Any file used in your project, such as an image, sound, model, or script.

Example: a player character is a GameObject with a Transform (where it is), a Sprite Renderer or Mesh Renderer (how it looks), a Collider (its shape for hits), and a Rigidbody (physics).

6. Where to go next

- Explore the editor: create a few objects, select them, and change their values in the Inspector.
- When you are ready, move on to scripting with C#.

7. Creating blue chicken with physics

In this section you will create a blue chicken, give it physics with a Rigidbody 2D and a Collider, and then put your own image on it.

Create the circle

In the Hierarchy, right-click and choose **2D Object** → **Sprites** → **Circle**. A white circle appears in the Scene view.

Optionally rename it (for example "blue chicken") at the top of the Inspector.

Add a Rigidbody 2D

With the circle selected, click **Add Component** at the bottom of the Inspector.

Search for **Rigidbody 2D** and click it.

Add a Circle Collider 2D

Click **Add Component** again.

Search for **Circle Collider 2D** and click it. A thin green outline appears around the circle in the Scene view.

Tip: In a 2D project, always pick the components with "2D" in the name. The versions without it only work in 3D projects.

What are they?

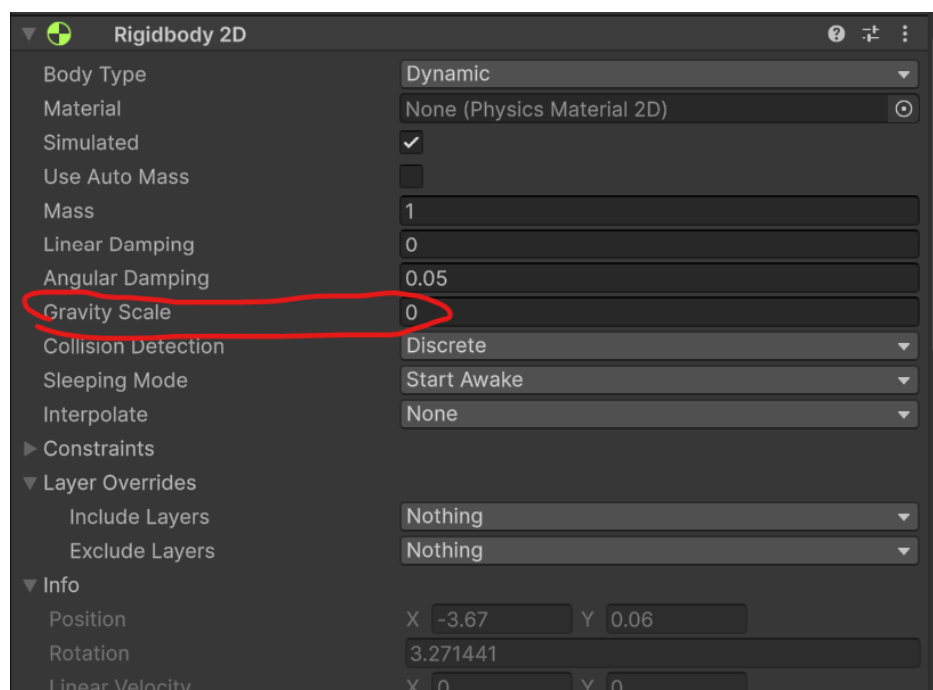
Rigidbody 2D hands the object over to Unity's physics system. Gravity pulls it down, forces can push it, and it can bounce off other objects. Without it, an object just stays where you put it.

Collider is an invisible shape that marks the solid edge of the object. Unity uses it to know when two objects touch. A Rigidbody with no Collider falls straight through everything.

You normally need both: the Rigidbody makes the object move, and the Collider makes it solid.

Test it

Press Play and the circle falls, because gravity now acts on it. It keeps falling because nothing is below it. Change the gravity to 0 to make it stay where it is



Add your image

The Sprite Renderer is the component that draws the picture. Your circle already has one, and you will swap its white circle image for your own.

- Drag your image file (PNG or JPG) from your computer into the **Project window**.
- Select the circle in the Hierarchy and find the **Sprite Renderer** in the Inspector.
- Drag your image from the Project window into the **Sprite** field. You can also click the small circle button at the right of that field and pick it.
- The image replaces the white circle. If it looks too big or too small, change the **Scale** in the Transform.



8. Making the blue chicken move

Now you will write a small script that moves the circle to the right. Scripts are written in C#.

The script

This script sets the object's velocity directly, so it moves at a steady speed.

```

using UnityEngine;

public class PlayerMovement : MonoBehaviour
{
    private Rigidbody2D rb;
    public float speed = 5;

    // Start is called once before the first execution of Update after the
    MonoBehaviour is created
    void Start()
    {
        rb = GetComponent<Rigidbody2D>();
    }

    // Update is called once per frame
    void Update()
    {
        rb.linearVelocity = new Vector2(1, 0) * speed;
    }
}

```

How to attach the script

4. In the **Project window**, right-click inside the Assets folder and choose **Create** → **Scripting** → **MonoBehaviour Script**. The menu wording can differ slightly between Unity versions.
5. Name it **PlayerMovement**.
6. Double-click the script to open your code editor. Replace everything inside with the code above and save the file.
7. Go back to Unity and wait a moment while it compiles the script.
8. Drag the script from the Project window onto the circle in the Hierarchy. You can also select the blue chicken, click **Add Component**, and search for PlayerMovement.
9. Select the blue chicken. In the Inspector you should now see a **Player Movement (Script)** component with the **Speed** field.
10. Press **Play**. The circle moves to the right. Try changing **Speed** in the Inspector to see the difference.

What each part of the code does

Code	What it does
<code>using UnityEngine;</code>	Lets your script use Unity's tools, such as Rigidbody2D, Vector2, and MonoBehaviour. Almost every Unity script starts with this line.
<code>public class PlayerMovement : MonoBehaviour</code>	Creates your script. The name must match the file name exactly. MonoBehaviour is what lets the script be attached to a GameObject and be run by Unity.

Code	What it does
<code>public Rigidbody2D rb;</code>	A variable that will hold the object's Rigidbody 2D. Because it is public, it shows up in the Inspector.
<code>public float speed = 5;</code>	A number for how fast the object moves. It is public, so you can change it in the Inspector without editing the code. The value in the Inspector wins over the 5 written here.
<code>void Start()</code>	Runs once, when the game starts. Good for setting things up.
<code>rb = GetComponent<Rigidbody2D>();</code>	Finds the Rigidbody 2D on the same object and stores it in rb. This means you do not have to fill in the rb slot yourself.
<code>void Update()</code>	Runs over and over, once every frame, while the game is playing.
<code>rb.linearVelocity = new Vector2(1, 0) * speed;</code>	Sets how fast the object is moving. new Vector2(1, 0) means "1 to the right, 0 up or down". Multiplying by speed makes that 5, so the object moves right at 5 units per second.
<code>rb.linearVelocity = new Vector2(speed, rb.linearVelocity.y);</code>	